week-2

MON - TED

1

INFORMATIONS

https://dusie.ch/topics/p5js/?c=setup

https://p5js.org/libraries/directory/

P5LIVE offline:

go to terminal

-> cd /Users/iarat/Documents/P5LIVE

npm start

to end

-> control C or close terminal

chrome better/faster - just one tab

(ohmyzsh - terminal shortcuts etc)

SHORTCUTS P5LIVE

option + shift + S edit snippets

control + shift + S quickly choose snippets

control + E hide/unhide code

control + M hide/unhide menu

control + N - new file

P5LIVE-Version

```
//version of p5
p5 = "1.4.0"

function setup() {
    createCanvas(windowWidth, windowHeight)
```

```
}

function draw() {

}
```

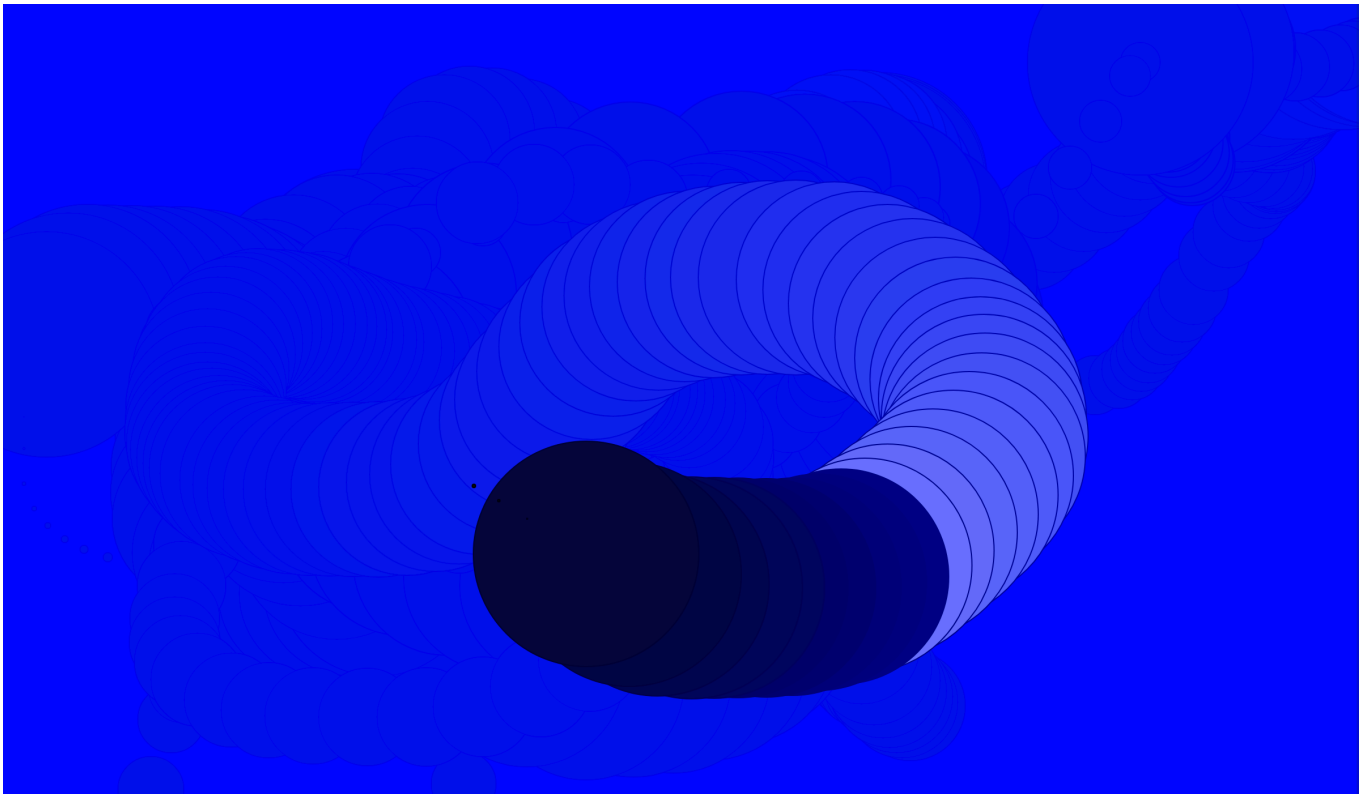
2

control + shift + A = audioreactive snippet

```
function setup() {
     createCanvas(windowWidth, windowHeight)

     setupAudio(true) // global vars
     // a5.ease = .075 // set easing
}

function draw() {
     /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */
     updateAudio()
     circle(mouseX, mouseY, ampEase*5)

}
```

3

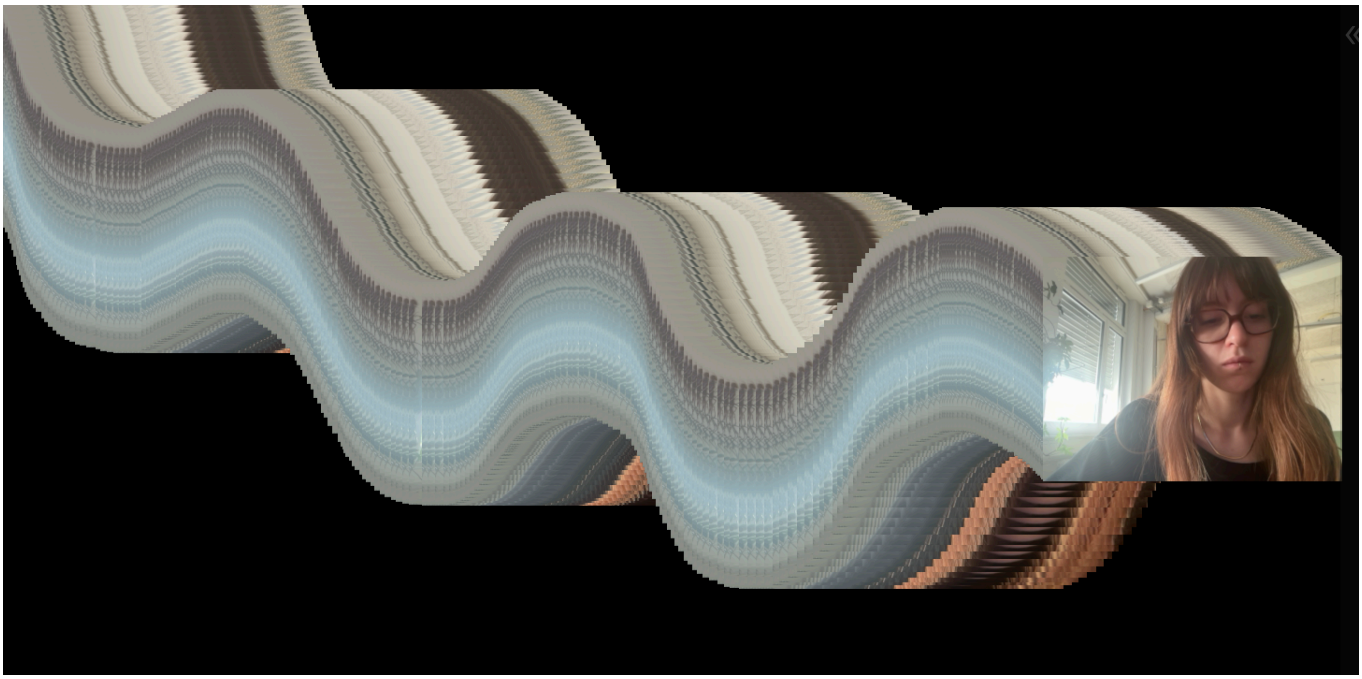
recoding something



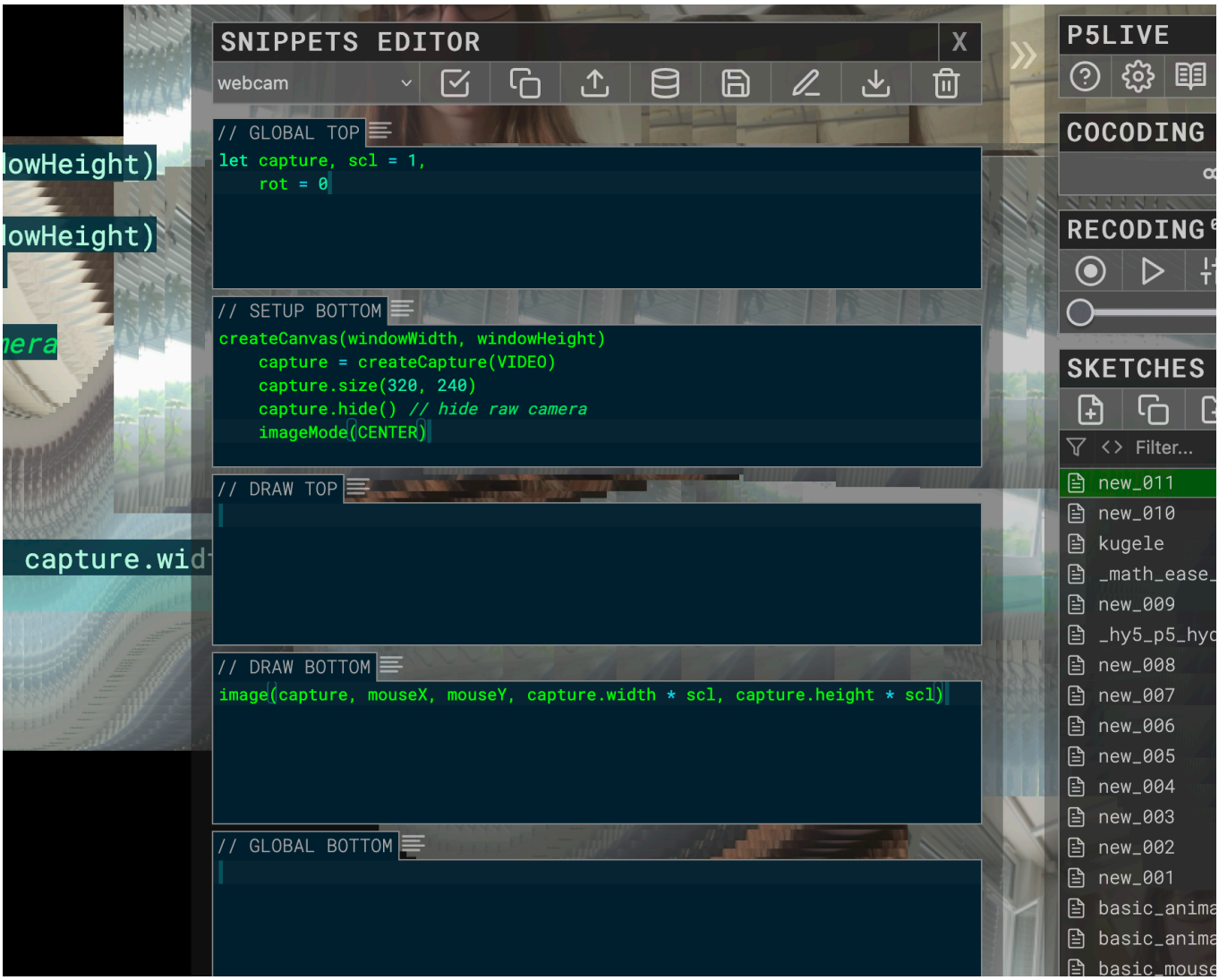
```
function setup() {
     createCanvas(windowWidth, windowHeight)

}

function draw() {
     background(0, 15, 255, 15)
     fill(frameCount % 255)
     circle(mouseX, mouseY, frameCount % 200)
}
```

4

how to create new snippet and edit it

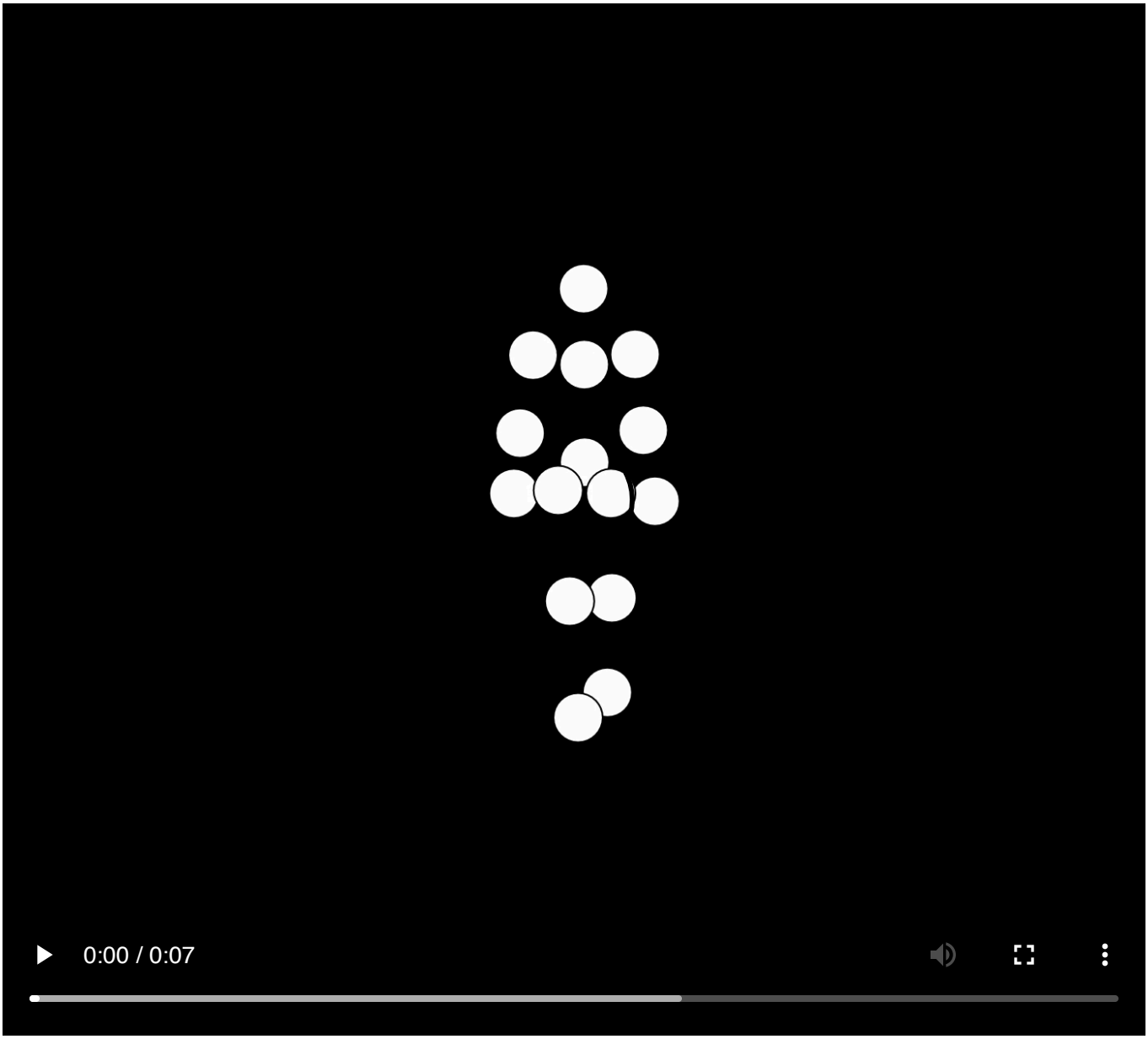


```
// {"P5LIVE":{"name":"new_011","mod":1777910514725}}

let capture, scl = 1,
     rot = 0

function setup() {
     createCanvas(windowWidth, windowHeight)

     createCanvas(windowWidth, windowHeight)
     capture = createCapture(VIDEO)
     capture.size(320, 240)
     capture.hide() // hide raw camera
     imageMode(CENTER)
}

function draw() {
scl = frameCount*0,01%1
     image(capture, mouseX, mouseY, capture.width * scl, capture.height *
scl)
}
```



5

import a library



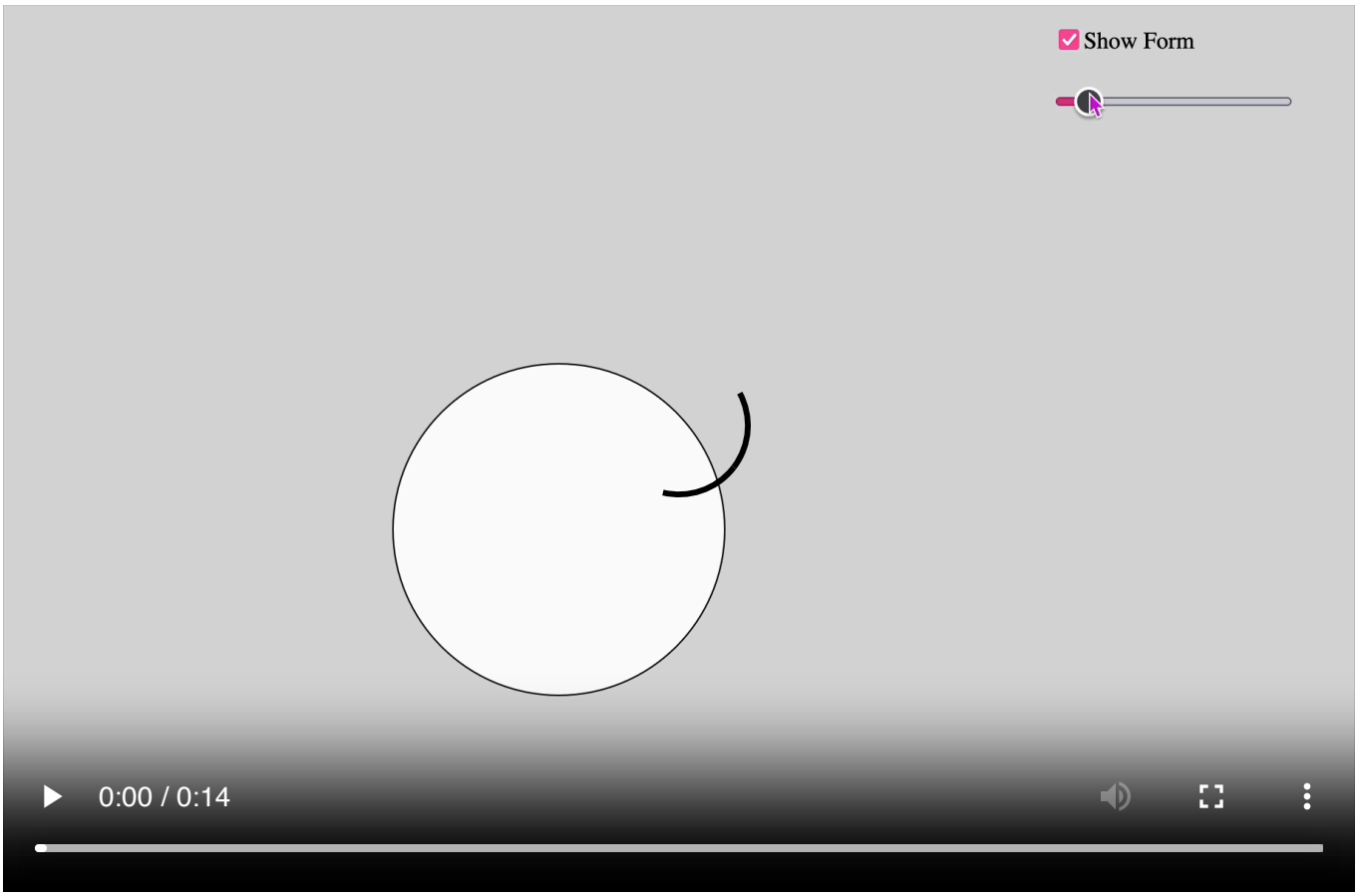

```
// {"P5LIVE":{"name":"new_010","mod":"1777910453842"}}

let libs =
['https://tetunori.github.io/BMWalker.js/dist/v0.6.1/bmwalker.js','']

// Create biological motion walker instance
const bmw = new BMWalker();

function setup() {
    createCanvas(windowWidth,windowHeight)

}

function draw() {
    clear()
```

```
// Get array of the current marker coordinates
const walkerHeight = 300;
const markers = bmw.getMarkers(walkerHeight);

translate (width /2, height/2)
     // Draw each markers
markers.forEach((m) => {
  circle(m.x, m.y, 30);
});
}
```

TUE - ALPER

1

slider




```
let slider, checkbox, button, colorPicker, dropdown, input, sliderText;

function setup() {
   createCanvas(windowWidth, windowHeight);

   //checkbox
   checkbox = createCheckbox('Show Form', true);
```

```
    checkbox.position(width - 400, 20);

    //sliders
    slider = createSlider(50, height - 100, 200);
    slider.position(width - 400, 60);
}

function draw() {
    background(220);

    if(checkbox.checked()) {
        ellipse(width / 2, height / 2, slider.value(), slider.value());
    }
}
```

2

button

```
let slider, checkbox, button, colorPicker, dropdown, input, sliderText;
let bgColor;
let positionDOM;

function setup() {

    positionDOM = width - 400

    // Button
    button = createButton('Random Background');
    button.position(positionDOM, 100);
    button.mousePressed(() => {
        bgColor = color(random(255), random(255), random(255));
    });
    bgColor = color(220);
}

function draw() {
    background(bgColor);
}
```

3

colorPicker

```
function setup() {

   // Color Picker
 colorPicker =      createColorPicker('#ff0000');
 colorPicker.position(positionDOM, 140);
}

function draw() {
  fill(colorPicker.value());
}
```

4

dropDown

```
function setup() {
   // Dropdown
   dropdown = createSelect();
   dropdown.position(positionDOM, 180);
   dropdown.option('Circle');
   dropdown.option('Square');
   rectMode(CENTER);
}

function draw() {

   if (checkbox.checked()) {
      if (dropdown.value() === 'Circle') {
         ellipse(width / 2, height / 2, slider.value(), slider.value());
      } else if (dropdown.value() === 'Square') {
         rect(width / 2, height / 2, slider.value(), slider.value());
      }
   }
}
```

5

Input


```
function setup() {
   // Input field
   input = createInput('Type text');
   input.position(positionDOM, 220);
   textAlign(CENTER, CENTER);
}

function draw() {

   if (checkbox.checked()) {
      if (dropdown.value() === 'Circle') {
         ellipse(width / 2, height / 2, slider.value(), slider.value());
      } else if (dropdown.value() === 'Square') {
          rect(width / 2, height / 2, slider.value(), slider.value());
      }
}

   fill(0)
   textSize(20);
   text(input.value(), width / 2, height / 2);
}
```

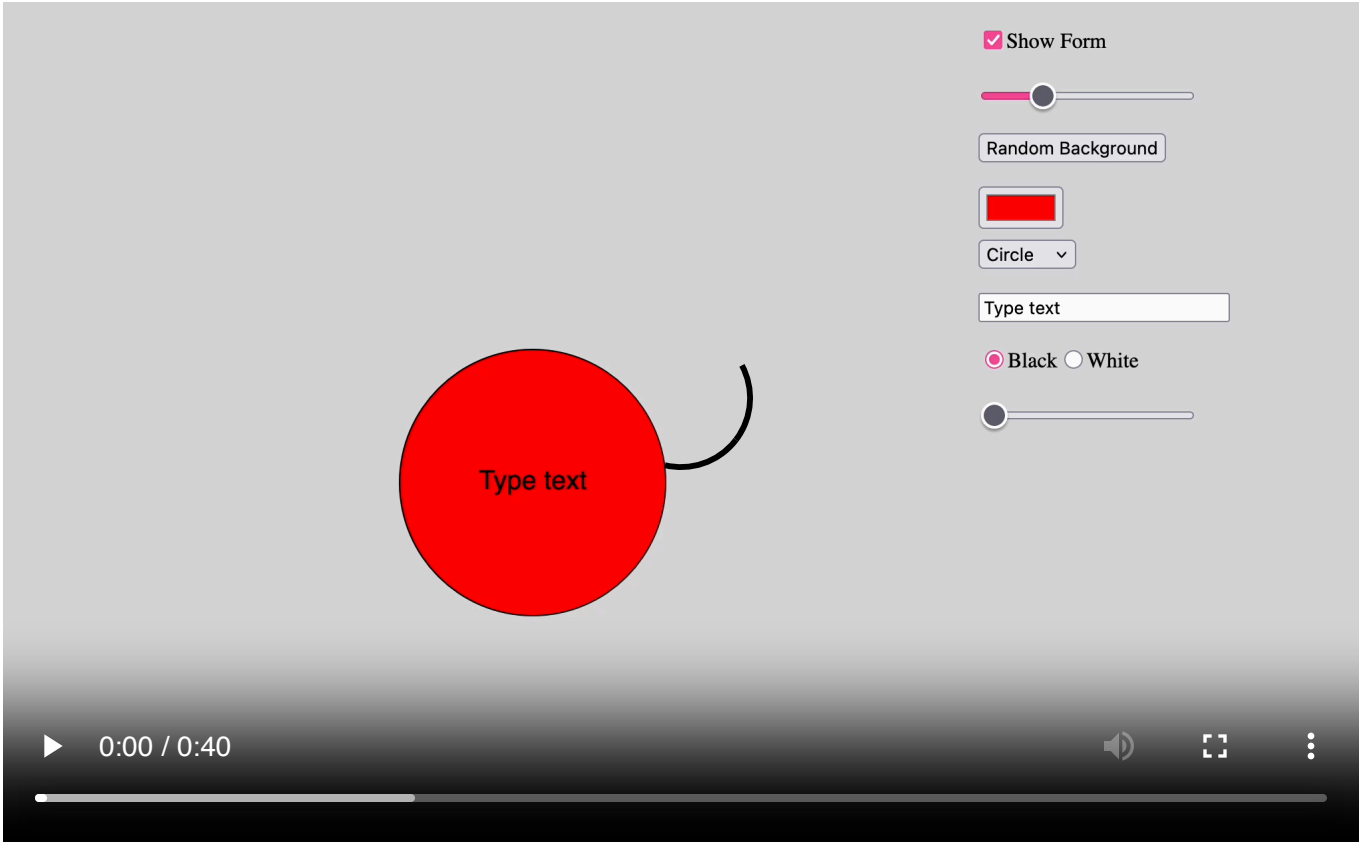
6

RadioButton

```
function setup() {
   //Radio button
   radio = createRadio();
   radio.option('Black');
   radio.option('White');
   radio.selected('Black');
   radio.position(positionDOM, 260);
}

function draw() {

    if (checkbox.checked()) {
        if (dropdown.value() === 'Circle') {
           ellipse(width / 2, height / 2, slider.value(), slider.value());
        } else if (dropdown.value() === 'Square') {
           rect(width / 2, height / 2, slider.value(), slider.value());
        }
}

    if (radio.value() === 'Black') fill(0);
    if (radio.value() === 'White') fill(255);

    textSize(20);
    text(input.value(), width / 2, height / 2);
}
```

7

all together

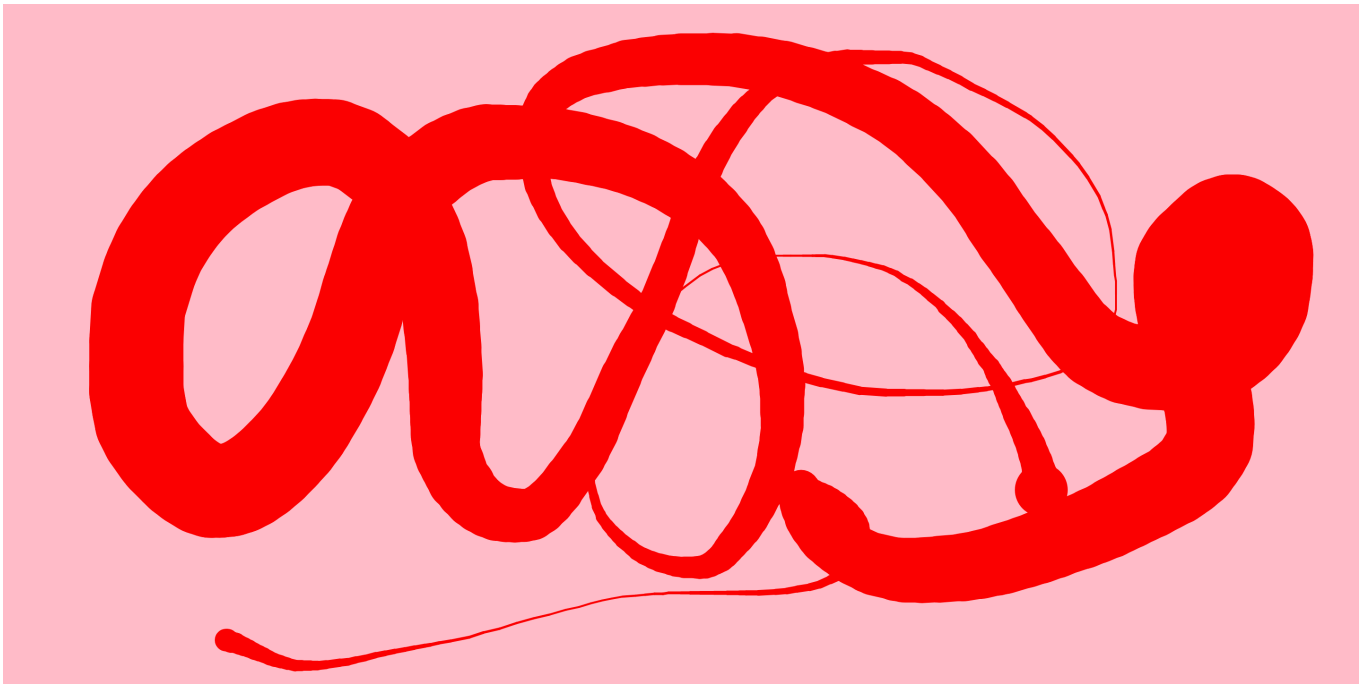



```
let slider, checkbox, button, colorPicker, dropdown, input, sliderText;
let bgColor;
let positionDOM;

function setup() {
    createCanvas(windowWidth, windowHeight);

    positionDOM = width - 400

    //checkbox
    checkbox = createCheckbox('Show Form', true);
    checkbox.position(positionDOM, 20);

    //sliders
    slider = createSlider(50, height - 100, 200);
    slider.position(positionDOM, 60);

    // Button
    button = createButton('Random Background');
    button.position(positionDOM, 100);
    button.mousePressed(() => {
       bgColor = color(random(255), random(255), random(255));
    });
    bgColor = color(220);
```

```
    // Color Picker
    colorPicker = createColorPicker('#ff0000');
    colorPicker.position(positionDOM, 140);

    // Dropdown
    dropdown = createSelect();
    dropdown.position(positionDOM, 180);
    dropdown.option('Circle');
    dropdown.option('Square');
    rectMode(CENTER);

    // Input field
    input = createInput('Type text');
    input.position(positionDOM, 220);
    textAlign(CENTER, CENTER);

    //Radio button
    radio = createRadio();
    radio.option('Black');
    radio.option('White');
    radio.selected('Black');
    radio.position(positionDOM, 260);

    sliderText = createSlider(20, 300, 20);
    sliderText.position(positionDOM, 300);
}

function draw() {
    background(bgColor);
    fill(colorPicker.value());

    if (checkbox.checked()) {
       if (dropdown.value() === 'Circle') {
          ellipse(width / 2, height / 2, slider.value(), slider.value());
       } else if (dropdown.value() === 'Square') {
          rect(width / 2, height / 2, slider.value(), slider.value());
       }
}

    if (radio.value() === 'Black') fill(0);
    if (radio.value() === 'White') fill(255);

    textSize(sliderText.value());
    text(input.value(), width / 2, height / 2);
}
```

8

simple Painteditor



```
function setup() {
  createCanvas(windowWidth, windowHeight);
background(200);
  stroke(0);
}

function draw() {

  let pen1 = map(sin(frameCount*0.025),-1,1,2,100)
  if (mouseIsPressed) {
    stroke(0);
    strokeWeight(pen1);
    line(prevX, prevY, mouseX, mouseY);
    // ellipse(mouseX,mouseY, pen1, pen1)
  }
  prevX = mouseX;
  prevY = mouseY;
}

function keyPressed(){
     if(key == 'S'){
          save('drawing.png')
     }
}
```

9

simple mouse and key editor

```
let moveX = 0;
let moveY = 0;
let position = 0;

function setup () {
  createCanvas(windowWidth, windowHeight);
}

function draw() {
  background(155)

  if (keyIsDown(49)) { // 49 = Taste "1"
    moveX = map(mouseX, 0, width, 0, 255);
    moveY = map(mouseY, 0, height, 0, 255);
  }

  if (keyIsDown(39)) {
    position += 2;
  }
  if (keyIsDown(37)) {
    position -= 2;
  }

  fill(255, 0, 0);
  ellipse(width / 2 + position, height / 2, 50 + moveX, 50 + moveY);

  text("keyCode: " + keyCode, 50, 100);
}
```

10

code snippet paint-editor

```
let strokeSize = 1;
let pg;

function setup() {
  createCanvas(windowWidth, windowHeight);
  pg = createGraphics(windowWidth, windowHeight);
  background(200);
}
```

```
function draw() {
  background(200);
  image(pg, 0, 0);

  strokeSize = constrain(strokeSize, 1, 300);

  // ZEICHNEN AUF pg
  if (mouseIsPressed) {
    pg.strokeWeight(strokeSize);
    pg.noFill();

    // Wenn eine Form-Taste gedrückt ist
      pg.push();
      pg.translate(mouseX, mouseY);

      if (key === 'a') {
        pg.stroke(255, 0, 0);
        pg.line(-25, -25, 25, 25);
        pg.line(25, -25, -25, 25);

      } else if (key === 's') {
        pg.stroke(0, 0, 255);
        pg.rect(-25, -25, 50, 50);

      } else if (key === 'd') {
        pg.stroke(255, 0, 255);
        pg.triangle(0, -25, -25, 25, 25, 25);

      } else if (key === 'f') {
        pg.stroke(255, 255, 0);
        pg.ellipse(0, 0, 50, 50);

      } else if (key === 'g') {
        pg.fill(200);
        pg.noStroke();
        pg.ellipse(0, 0, 50, 50);
      }

      pg.pop();

    }


  // VORSCHAU (nur anzeigen, nicht speichern)
  if (keyIsPressed && !mouseIsPressed) {
```

```
    strokeWeight(strokeSize);
    noFill();

    push();
    translate(mouseX, mouseY);

    if (key === 'a') {
      stroke(255, 0, 0, 150);
      line(-25, -25, 25, 25);
      line(25, -25, -25, 25);

    } else if (key === 's') {
      stroke(0, 0, 255, 150);
      rect(-25, -25, 50, 50);

    } else if (key === 'd') {
      stroke(255, 0, 255, 150);
      triangle(0, -25, -25, 25, 25, 25);

    } else if (key === 'f') {
      stroke(255, 255, 0, 150);
      ellipse(0, 0, 50, 50);
    }

    pop();
  }
}

function mouseWheel(event) {
  strokeSize += event.delta / 2;
}

function keyPressed(){
    if(key == 'S'){
        save('drawing.png')
    }
}
```

WED - ANDREA

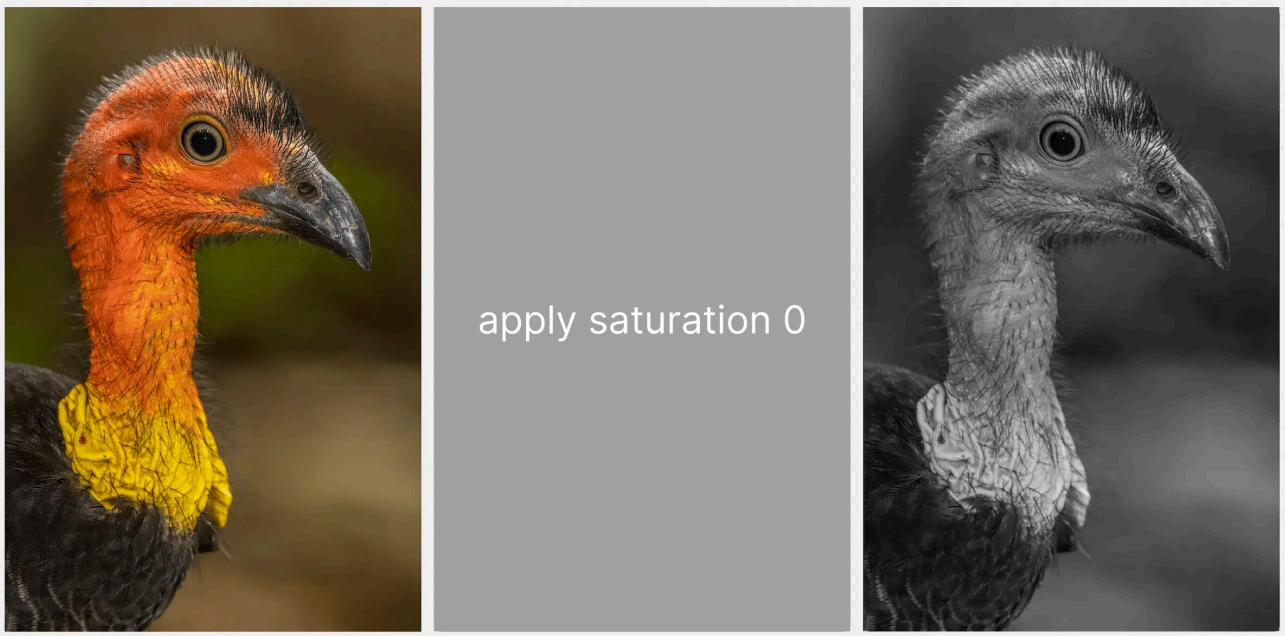
https://learn-andreazaccuri.azeta.workers.dev/260506-colabor/sources/

hydra

Hydra is a live-coding environment for visuals, built on top of WebGL and inspired by analog video synthesizers.

how it works

Hydra works by *chaining* a source with transformations, and finally rendering the output on screen.



feedback loops

A feedback loop is what happens when the **output** of a chain becomes the **input** of the next frame. In Hydra this is just `src(o0)` — read from the screen you just drew. Once you have that, small perturbations compound into rich, evolving textures. That's what we'll explore today.

sources

solid

Fills the whole screen with one flat color. You pass red, green and blue values from 0 to 1.

```
solid(1, 0, 0).out()       // pure red
solid(0, 1, 0).out()       // pure green
solid(0.5, 0.5, 1).out()   // light blue
```

shape

Draws a polygon in the middle of the screen. You decide how many sides it has, how big it is and how soft the edges look.

```
shape().out()
shape(3).out()       // triangle
shape(4).out()       // square
shape(6).out()       // hexagon
shape(100).out()     // so many sides it looks like a circle
shape(4, 0.5).out()  // bigger square
shape(4, 0.5, 0.2).out() // square with soft edges
```

gradient

A rainbow gradient that covers the whole screen. An optional parameter controls how fast the colors shifts over time.

```
gradient().out()
gradient(1).out()  // colors shifts over time
```

osc

Makes (colored) stripes that scroll across the screen.

```
osc(20, 0.1, 0.8).out()
//   ↑    ↑    ↑
//   |    |    └── how colorful it is (0 = grayscale, higher = more colors)
//   |    └─── how fast it moves (0 = still, higher = faster)
//   └─── how many stripes (small = large, large = thin)
```

noise

Random clouds of light and dark spots. You can change how big the clouds are and how fast they move.

```
noise().out()
noise(3).out()      // bigger
noise(20).out()     // smaller, more detailed
noise(10, 1).out()  // moves faster
```

voronoi

Cells that look like the skin of a giraffe. You can change the size, the speed and how blurry they are.

```
voronoi().out()
voronoi(5).out()        // big
voronoi(20).out()       // small
voronoi(10, 0).out()    // slow
voronoi(10, 1, 1).out() // blurry
```

from hydras memory (buffers)

Hydra has four buffers, called o0, o1, o2, and o3. Immagine them as pieces of paper.

So far we have been using only the buffer o0, which is the default when you write `.out()`.

Now we will learn how to make Hydra remember what he draws on the first piece of paper, and make it output on a different one.

```
osc(20).out(o0)
noise(3).out(o1)
render(o1)
```

(here one on top of each other, so u cant see both)

src

With `src()` you can be specific about which buffer to use.

```
osc(20).out(o0)
src(o0).rotate(0.5).out(o1)
render(o1)
```

! the four buffers !

You can draw on all four pieces of paper at the same time. With `render()` you see all of them in a grid.

```
shape(3).out(o0)        // buffer 0
osc(30, 0.1, 2).out(o1) // buffer 1
gradient(2).out(o2)     // buffer 2
noise(10, 1).out(o3)    // buffer 3
render()
```

“/Users/iarat/Desktop/dd+a/colabor_creative_coding_2026/documentation/images/4_buffers.png” konnte nicht gefunden werden.

we can play with that, put one on top of each other and tell output which layer you want to see

external sources

Images

Loads a picture from a link on the internet and uses it as a source.

```
s0.initImage("https://upload.wikimedia.org/wikipedia/commons/2/25/Hydra-Foto.jpg")
src(s0).out()
```

Videos

Loads a video from a link on the internet and plays it as a source.

```
s0.initVideo("https://upload.wikimedia.org/wikipedia/commons/8/86/A_Boy_and_His_Dragon.webm?utm_source=commons.wikimedia.org&utm_campaign=index&utm_content=original")
src(s0).out()
```

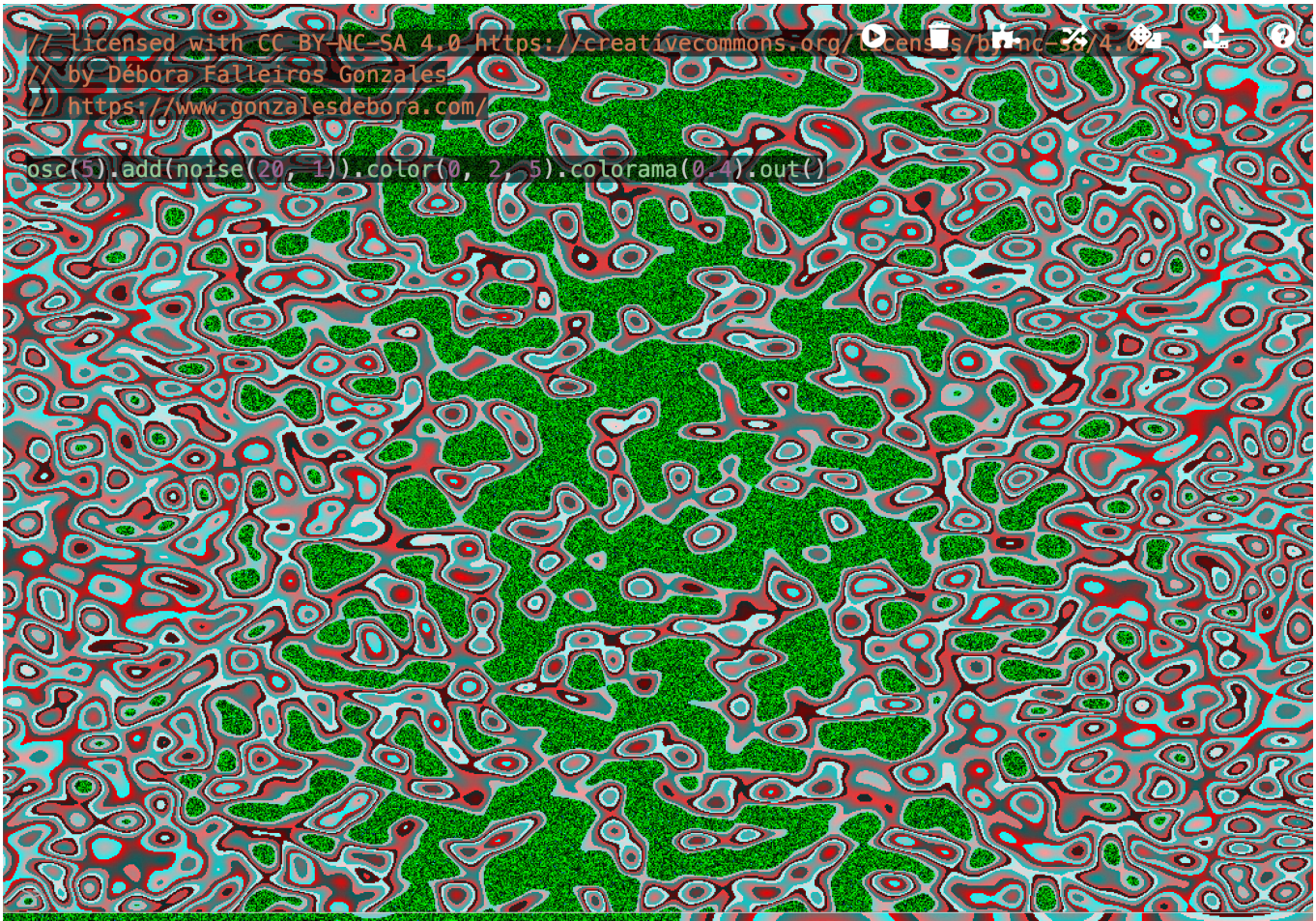
Camera

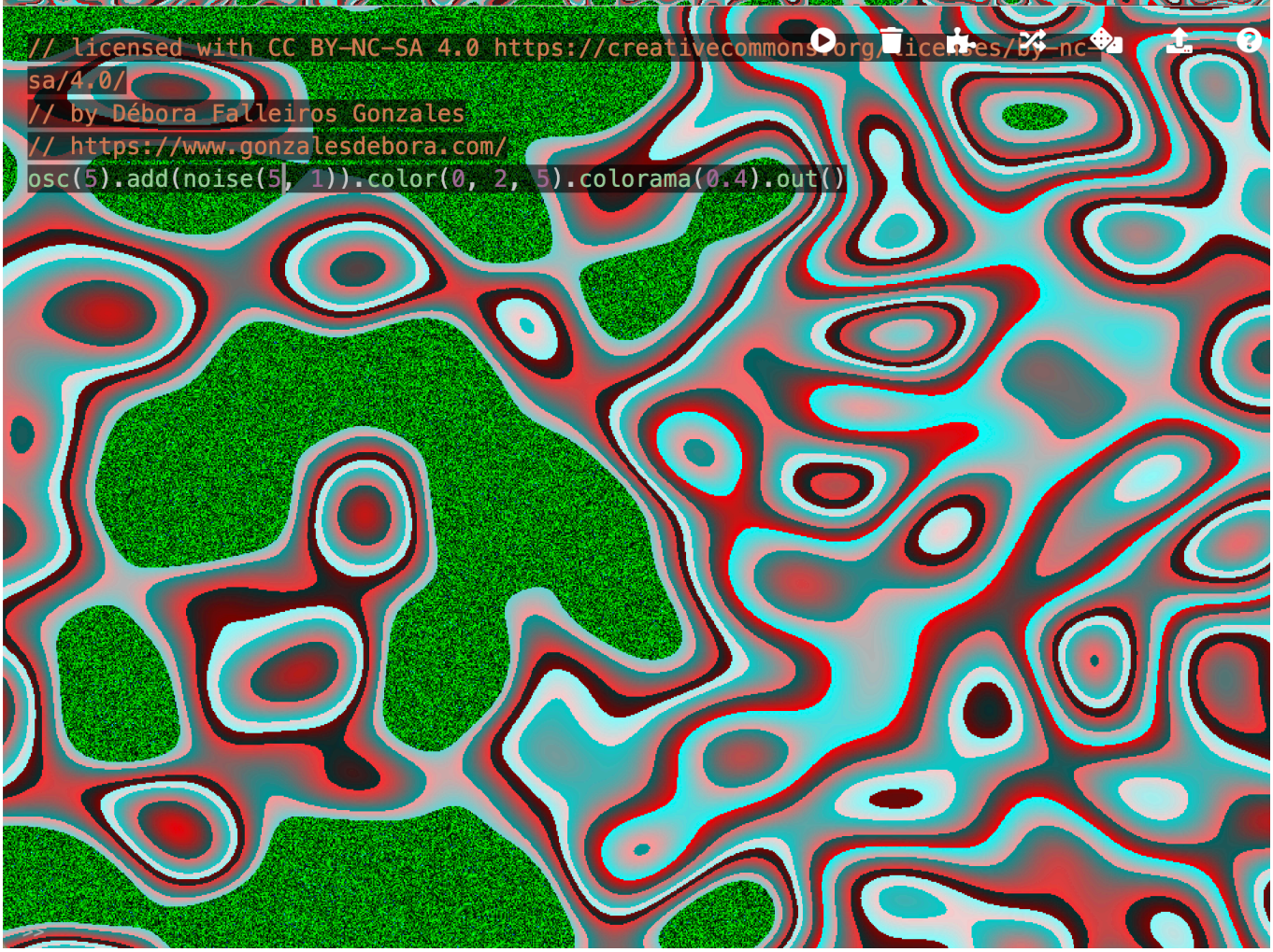
Uses your webcam as a source. Your browser will ask for permission the first time.

```
s0.initCam()
src(s0).out()
```

experiment

1


// licensed with CC BY-NC-SA 4.0 https://creativecommons.org/licenses/by-nc-sa/4.0/
// by Débora Falleiros Gonzales
// https://www.gonzalesdebora.com/
osc(5).add(noise(20, 1)).color(0, 2, 5).colorama(0.4).out()


// licensed with CC BY-NC-SA 4.0 https://creativecommons.org/licenses/by-nc-sa/4.0/
// by Débora Falleiros Gonzales
// https://www.gonzalesdebora.com/
osc(5).add(noise(5, 1)).color(0, 2, 5).colorama(0.4).out()

```
osc(5).add(noise(20, 1)).color(0, 2, 5).colorama(0.4).out()
```

transforms

a transform comes after a source

there is no limit on how many transformation you can apply to a visual

Each transform takes a pixel and defines how it changes, passing it to the next transformation, until it reaches the out() function. We will explore three families of transformations: geometry, colors, channels.

geometry

rotate

Spins the image around its center. The number is the angle in radians.

```
osc(20).rotate(10).out();
```

scale

Makes the image bigger or smaller. Numbers below 1 zoom out, numbers above 1 zoom in.

```
osc(20).scale(.1).out();
```

pixelate

Breaks the image into big blocks of color.

```
osc(20).pixelate(10).out();
```

repeat

Tiles the image many times across the screen. You can repeat in both directions, or only on X or only on Y.

```
shape(3).repeat(2, 2).out();
```

repeatX just on x -> good for offset

reapeatY

kaleid

mirrors the image like a kaleidoscope

```
osc(20).kaleid(4).out();
```

scroll

Slides the image across the screen. You can scroll on Y (up and down) or on X (left and right).

```
shape(3).scrollY(1, .75).out();
```

scrollX

colors

invert

```
noise(20, .5).invert(1).out();
```

brightness

```
osc(20).brightness().out();
```

contrast

```
osc(20).contrast().out();
```

saturate

```
osc(20).saturate().out();
```

hue

```
osc(20).hue().out();
```

colorama

Pushes the colors into psychedelic shifts.

```
osc(20).colorama().out();
```

posterize

Reduces the image to a few flat colors.

```
osc(20).posterize().out();
```

tresh

Turns the image into pure black and white based on how bright each pixel is.

```
osc(20).thresh().out();
```

luma

Keeps only the bright parts of the image and hides the dark ones (they become transparent).

```
osc(20).luma().out();
```

shift

Moves each color channel separately.

```
osc(20).shift().out();
```

color

Multiplies the red, green and blue channels. Useful to tint the image.

```
osc(20).color().out();
```

layering

composition

Combining two or more chains, making them interact and creating complex effects.

color blending

add

A + B (brighter where both are bright)

```
osc(20).add(noise(3)).out()
osc(20).add(noise(3), 0.5).out()
```

substract

A − B (darker where second is bright)

```
osc(6,0,1.5).modulate(noise(3).sub(gradient()),1).out(o0)
```

multiply

A × B (only bright where both are bright; perfect for masking)

```
osc(20).mult(noise(3)).out()
```

difference

A − B (bright where they’re different)

```
osc(20).diff(noise(3)).out()
```

blend

average of A and B

```
osc(20).blend(noise(3)).out()
osc(20).blend(noise(3), 0.3).out()    // mostly osc, a bit of noise
osc(20).blend(noise(3), 0.7).out()    // mostly noise, a bit of osc
```

layer

B if B isn’t transparent, otherwise A

```
shape(4).layer(osc(20).luma(0.5).out()).out()
```

mask

A, but only visible where B is bright

```
osc(20).mask(shape(4)).out()
```

modulation

Modulation uses one chain's brightness to push the other chain's pixels around. One chain becomes a force that distorts the other.

In bleeding we were able to see both chains interacting, in modulation we only see the modulated chain.

There are eleven modulate* functions. They all work the same way: the second chain is the "pusher." What changes between them is what gets pushed.

modulate

```
osc(20).modulate(noise(3), 0.3).out()
```

modulate rotate

```
osc(20).modulateRotate(noise(3), 1).out()
```

modulate scale

```
osc(20).modulateScale(noise(3), 0.5).out()
```

modulate repeat

```
osc(20).modulateRepeat(noise(3), 3, 3).out()
```

modulate repeat X Y

```
osc(20).modulateRepeatX(noise(3), 3, 3).out()
```

```
osc(20).modulateRepeatY(noise(3), 3, 3).out()
```

modulate kaleid

```
osc(20).modulateKaleid(noise(3), 4).out()
```

modulate scroll X Y

```
osc(20).modulateScrollX(noise(3), 0.5).out()
```

```
osc(20).modulateScrollY(noise(3), 0.5).out()
```

modulate pixelate

```
osc(20).modulatePixelate(noise(3), 30).out()
```

modulate heu

```
osc(20).modulateHue(noise(3), 1).out()
```

experiment


```
osc(20).modulatePixelate(noise(5), 10).colorama().out(o1)
```

outputs

building feedback

We can run two or more Hydra sketches at once

```
osc(20, 0.1, 1.5).kaleid(4).out(o0)
noise(3).pixelate(20, 20).out(o1)

render(o0)    // show stripes
render(o1)    // show noise
```

Or use the output of one sketch as input to another, to create a feedback loop

```
osc(20, 0.1, .5).kaleid(4).out(o0)  // pattern on o0
src(o0).rotate(0.25).out(o1)        // use it on o1
render(o1)
```

arrays

Its possible to use an array instead of any value, hydra will cycle through the values during runtime.

```
osc([20, 3, 8]).out()
```

dynamic values

```
time      // seconds since the page loaded
mouse.x   // horizontal mouse position in pixels
mouse.y   // vertical mouse position in pixels
width     // canvas width in pixels
height    // canvas height in pixels
```

Important: to use these values in our sketches, we need them in an arrow function.

A variable like `time` needs to be re-evaluated at every frame to be used.

```
osc(() => time).out()
```

audio reactive sketches

audio analysis in hydra

Hydra exposes audio through an object called a. (short for *audio*).

```
a.show() // shows the audio spectrum
```

By default Hydra groups the audio into 4 bands:

```
a.fft[0]    // bass
a.fft[1]    // low-mids
a.fft[2]    // high-mids
a.fft[3]    // treble
```

The number of bands can be customized by passing a parameter to the `setBins()` method:

```
a.setBins(8)
```

We can also pass other controls, like smoothers or scaling functions:

```
a.setSmooth(0.8)
a.setScale(5)
```

Audio needs to be used in an arrow function:

```
osc(20).rotate(() => a.fft[0] * 3).out()
//              ↑
//              └── arrow function so the audio updates every frame
```

patterns

```
osc(20)
  .scale(() => 1 + a.fft[0])
  .out()
```

```
osc(40, 0.1, 1.5)
  .scale(() => 1 + a.fft[0] * 0.5)          // bass
  .modulate(noise(3), () => a.fft[1] * 0.3) // mids
  .kaleid(() => 3 + a.fft[2] * 5)           // high mids
  .hue(() => a.fft[3] * 0.5)                // treble
  .out()
```

```
osc(20)
  .modulate(
    noise(3, () => a.fft[0] * 2),     // noise speeds up with bass
```

```
    () => a.fft[2] * 0.3              // modulation strength tied to high-
mids
  )
  .out()
```

to put hydra in p5

```
/*
    _HY5_p5_hydra // cc teddavis.org 2024
    pass p5 into hydra
    docs: https://github.com/ffd8/hy5
*/

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

voronoi(8,1)
.mult(osc(10,0.1,()=>Math.sin(time)*3).saturate(3).kaleid(200))
.modulate(o0,0.5)
.add(o0,0.8)
.scrollY(-0.01)
.scale(0.99)
.modulate(voronoi(8,1),0.008)
.luma(0.3)
.out()


speed = 0.1


function setup() {
    createCanvas(windowWidth, windowHeight)

}

function draw() {
    //circle(mouseX,mouseY,100)
}
```

and in p5 noise -> noize

because in p5 already exist noise

experiment

1


```
/*
    _HY5_p5_hydra // cc teddavis.org 2024
    pass p5 into hydra
    docs: https://github.com/ffd8/hy5
*/

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5
```

```
  osc(20, 0.1,2.5)
  .modulate(src(o0).scale(0.95), 0.15).kaleid(4).colorama(1)
  .out(o0)
```

```
function setup() {
    createCanvas(windowWidth, windowHeight)

}

function draw() {
    //circle(mouseX,mouseY,100)
}
```

2


```
/*
    _HY5_p5_hydra // cc teddavis.org 2024
    pass p5 into hydra
```

```
    docs: https://github.com/ffd8/hy5
*/

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

 osc(5, 0.1, () => mouse.y * 0.1).modulatePixelate(noise(5), 5).colorama(5)

 .out()




function setup() {
    createCanvas(windowWidth, windowHeight)

}

function draw() {
    //circle(mouseX,mouseY,100)
}
```

3


```
/*
    _HY5_p5_hydra // cc teddavis.org 2024
    pass p5 into hydra
    docs: https://github.com/ffd8/hy5
*/

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

//color-luma

osc(20,0.08).luma(0.15,).color(1,4,0).colorama(10).kaleid(100)
.out()
```

```
function setup() {
    createCanvas(windowWidth, windowHeight)

}

function draw() {
    //circle(mouseX,mouseY,100)
}
```

4


```
/*
    _HY5_p5_hydra // cc teddavis.org 2024
    pass p5 into hydra
    docs: https://github.com/ffd8/hy5
*/

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']
```

```
// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

//color-luma

osc(20,0.08).luma(0.15).color(1,4,0).colorama(10)
.out()




function setup() {
    createCanvas(windowWidth, windowHeight)

}

function draw() {
    //circle(mouseX,mouseY,100)
}
```

JASMIN

https://nervousdata.com/

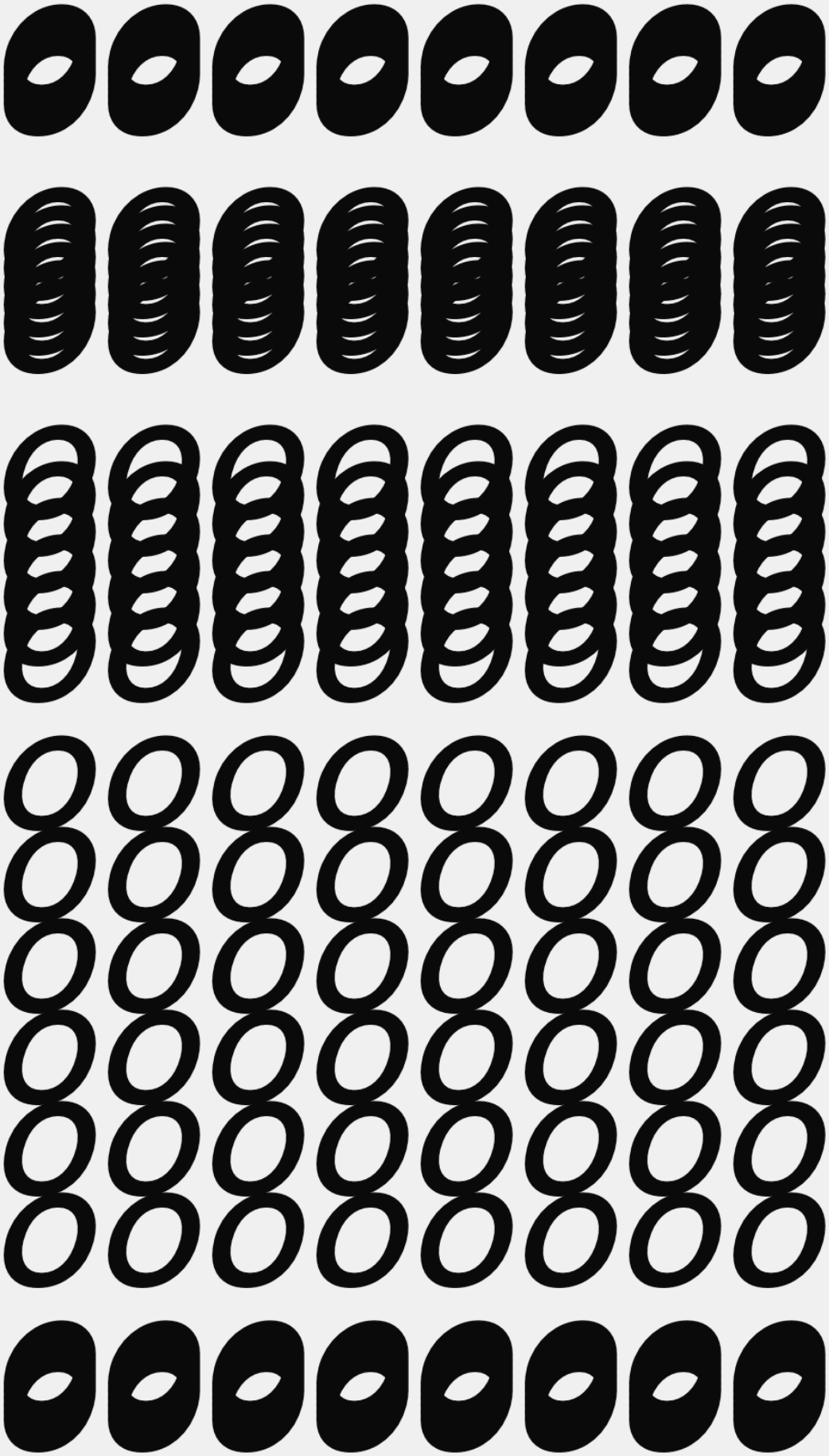
text

textWrap(WORD/CHAR)

word starts new lines of text at spaces. If a string of text doesn't have spaces, it may overflow the text box and the canvas. This is the default style.

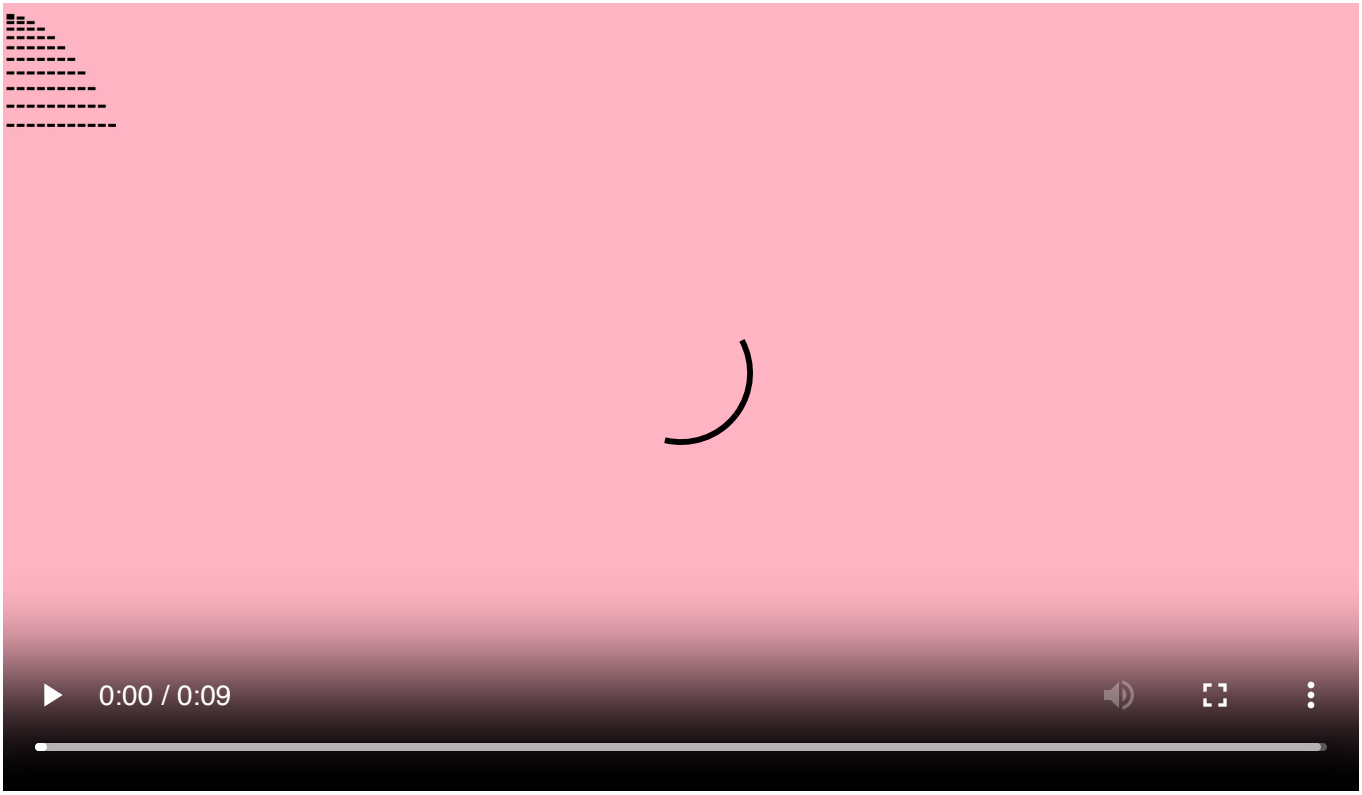
char starts new lines as needed to stay within the text box.

1



```
function setup() {
     createCanvas(windowWidth, windowHeight)

}

function draw() {
      background(240)
      fill(10)
      textSize(100)
      textWrap(CHAR)
      textFont("Favorit")
      textAlign(RIGHT)
      textStyle(ITALIC)
      textLeading(4)
      text("ooooooooooooooooooooooooooooooooooooooooooooooo",500,
0,windowWidth/3,windowHeight)
      textLeading(10)
      text("ooooooooooooooooooooooooooooooooooooooooooooooo",500,
100,windowWidth/3,windowHeight)
      textLeading(20)
      text("ooooooooooooooooooooooooooooooooooooooooooooooo",500,
230,windowWidth/3,windowHeight)
      textLeading(50)
      text("ooooooooooooooooooooooooooooooooooooooooooooooo",500,
400,windowWidth/3,windowHeight)
      textLeading(4)
      text("ooooooooooooooooooooooooooooooooooooooooooooooo",500,
720,windowWidth/3,windowHeight)
}
```

2




```
function setup() {
     createCanvas(windowWidth, windowHeight)
     frameRate(3)
     noSmooth()
}

function draw() {
      background("pink")
       let count = frameCount%10
       let comma = ","
       let space = ["-"]
       let size = 50
       let commaline;

       fill(0)
       textSize(windowWidth/50)
       textFont("Helvetica")
       textWrap(CHAR)
       for (let i=0; i<11; i++){
           space.push("-");
           comma = comma + space[i]
           commaline = comma.repeat(count)
           text(commaline, 100*count,
commaline.length*i/1.1,windowWidth/2.5,windowHeight)
       }
}
```

3


```
function setup() {
     createCanvas(windowWidth, windowHeight)

}

function draw() {
      let live = frameCount%8
      let sine = floor(5*sin(frameCount/10)+5)
      let words =
["aom","xxx","pla","525","www","999","ttt","lala","3221","plop"]
      let rand = random(words)

     frameRate(2)
      background(240)
      fill(random(230))
      textSize(100+(sine+5))
      textWrap(CHAR)
      textFont("Helvetica")
      textAlign(LEFT)
      textStyle(random([NORMAL,ITALIC]))
      textLeading(100)
      text(words[live].replace(/[ap2]/g, "**").repeat(100), 50,
```

```
0,windowWidth,windowHeight)
}
```

4


```
function setup() {
     createCanvas(windowWidth, windowHeight)

}

function draw() {
      let live = frameCount%8
      let sine = floor(5*sin(frameCount/10)+5)
      let words =
["aom","xxx","pla","525","www","999","ttt","lala","3221","plop"]
      let rand = random(words)

     frameRate(2)
      background(0,0,255,150)
      fill(255,150,0)
      textSize(100+(sine+5))
      textWrap(CHAR)
      textFont("zapfino")
      textAlign(LEFT)
      textStyle(NORMAL)
      textLeading(100)
      text(words[live].replace(/[ouiea]/g, "    ").repeat(100), 50,
0,windowWidth,windowHeight)
}
```

5


```
function setup() {
     createCanvas(windowWidth, windowHeight)
     frameRate(3)
     noSmooth()
}

function draw() {
      background(255,0,0,200)
       let count = frameCount%10
       let comma = "__"
       let space = ["  "]
       let size = 50
       let commaline;

       fill(0)
       textSize(windowWidth/50)
       textFont("Helvetica")
       textWrap(CHAR)
       for (let i=0; i<11; i++){
           space.push("<3");
           comma = comma + space[i]
           commaline = comma.repeat(count+1+33)
           text(commaline, 10, commaline.length,windowWidth,windowHeight)
```

```
    }
}
```

6


```
function setup() {
     createCanvas(windowWidth, windowHeight)
     frameRate(2)
     noSmooth()
}

function draw() {
      background("blue")
      let count = frameCount%10
      let comma = "aha"
      let space = [""]
      let size = 50
      let commaline;
      fill(255)
      textSize(windowWidth/50)
      textFont("Helvetica")
      textWrap(CHAR)
      for (let i=0; i<11; i++){
          space.push("oooo");
          comma = comma + space[i]
          commaline = comma.repeat(count+1)
          text(commaline, 100, commaline.length,windowWidth,windowHeight)
      }
```

```
}
```

PAULINA ZYBINSKA

machine learing in browser

- **ml5.js** high-level library
- **Teachable Machine** tool

often used in combination with p5

pre-trained models

body & face estimation

BodyPose

BodySegmentation

HandPose

FaceMesh

classification

ImageClassifier

SoundClassifier

natural language processing

Sentiment

spatial awareness

ObjectDetection

DepthEstimation

(ma vny tutorials and LLMs refer to an older version of ml5.js library found here. If you see some untypical errors in the console, this might be often the issue.)

teachable machine

web-based tool

allow you to "teach" computer to recognize things (facespecific, gestures, or certain sounds)

1 change the name of the classes to something that is understandable

2 add images (at least 20) or video frames (between 100 and 200) to each class

3 click on "Train Model"

4 after the model is trained click on "Export Model"

5 in the pop-up window select "Upload Model", and copy the link to your Teachable Machine model to use in p5Live

ressources

The Coding Train ml5.js Begginer's Guide

Patt Vira Coding Tutorials

ml5js Glossary

advanced web-based tools for ML

mediaPipe

ONNX runtime

Teachable Machine + P5live

```
let libs = ['https://unpkg.com/ml5@1/dist/ml5.min.js'];

let classifier;
 //replace with your link to Teachable Machine model
 // you can train your model here
https://teachablemachine.withgoogle.com/train/image
let imageModelURL =
'https://teachablemachine.withgoogle.com/models/m32dUhe3gq/';

let video;
let label = 'loading...';
let confidence = 0;

function preload() {
  classifier = ml5.imageClassifier(imageModelURL + 'model.json');
}

function setup() {
  createCanvas(windowWidth, windowHeight);
  video = createCapture(VIDEO, { flipped: true });
```

```
  video.size(640, 480);
  video.hide();

  classifyVideo();

  textFont('monospace');
  textAlign(CENTER, CENTER);
}

function draw() {
  background(0);

  image(video, 0,0, width, height);

  // Label
  noStroke();
  fill(0, 180);
  rect(0, height - 100, width, 100);

  fill(255);
  textSize(48);
  text(label, width / 2, height - 60);

  textSize(18);
  text(nf(confidence, 1, 2), width / 2, height - 20);
}

function classifyVideo() {
  classifier.classify(video, gotResult);
}

function gotResult(results) {
  label = results[0].label;
  confidence = results[0].confidence;
  classifyVideo();
}
```

HandPose

```
let libs = ['https://unpkg.com/ml5@1/dist/ml5.min.js'];

let video;
let state = 'starting...';
let handPose;
let hands = [];
```

```
let options = { maxHands: 2, flipHorizontal: true };
let pScale = 20

function preload() {
  handPose = ml5.handPose(options);
}

function setup() {
  createCanvas(windowWidth, windowHeight);

  video = createCapture(VIDEO, { flipped: true });
  video.size(width, height);
  video.hide();

  handPose.detectStart(video, gotHands);
  state = 'detecting hands';
}

function draw() {
  background(20, 100);


    for (let i = 0; i < hands.length; i++) {
      const hand = hands[i];
      for (let j = 0; j < hand.keypoints.length; j++) {
        const kp = hand.keypoints[j];
        const x =  kp.x;
        const y =  kp.y;
        fill(map(j, 0, hand.keypoints.length, 0, 255), 255, map(j, 0,
hand.keypoints.length, 255, 0));
        noStroke();
        circle(x, y, pScale * sin(frameCount * 0.1 + j) % pScale);
      }
    }

  noStroke();
  fill(255);
  text(state, 10, height - 10);
}

function gotHands(results) {
  hands = results;
}
```

object detection

```
let video;
let detector;
let detections = [];

function preload(){
  // Load the COCO SSD model
  // This model is trained on the COCO dataset, which contains 80 common
objects
  // https://github.com/tensorflow/tfjs-models/blob/master/coco-
ssd/src/classes.ts
  detector = ml5.objectDetection("cocossd");
}

function setup() {
  createCanvas(960, 540); //change to 640, 480 if using webcam
  background(0);

 // uncomment these lines if using webcam
  /* video = createCapture(VIDEO);
  video.size(width, height);
  video.hide();  */

  //uncomment these lines if using video file
   video = createVideo("/2026_05_08-
Paulina_Zybinska/code/objectDetection/assets/rainyday2.mp4");
  video.size(width, height);
  video.hide();
  video.loop();

  detector.detectStart(video, gotDetections);
}

// Callback function is called each time the object detector finishes
processing a frame.
function gotDetections(results) {
  detections = results;
}

function draw() {
  background(0);

  let scaleX = width / video.elt.videoWidth;
  let scaleY = height / video.elt.videoHeight;

  //extra (vanilla javascript) to create a clip path made of all detection
rectangles
```

```
  /*drawingContext.save();
  drawingContext.beginPath();
  for (let i = 0; i < detections.length; i++) {
    let d = detections[i];
    drawingContext.rect(
      d.x * scaleX,
      d.y * scaleY,
      d.width * scaleX,
      d.height * scaleY
    );
  }
  drawingContext.clip();*/

  image(video, 0, 0, width, height);

  //extra (vanilla javascript) to restore the drawing context after the clip
path is created
  //drawingContext.restore();

  // outlines + labels on top
  for (let i = 0; i < detections.length; i++) {
    let d = detections[i];
    let x = d.x * scaleX;
    let y = d.y * scaleY;
    let w = d.width * scaleX;
    let h = d.height * scaleY;

    stroke(0, 255, 0);
    strokeWeight(2);
    noFill();
    rect(x, y, w, h);

    noStroke();
    fill(255);
    textSize(24);
    text(d.label, x + 10, y + 24);
  }
}
```

p5live-ml5.js

```
let libs = ['https://unpkg.com/ml5@1/dist/ml5.min.js'];

let video;
let state = 'starting...';
```

```
let faceMesh;
let faces = [];
let options = { maxFaces: 1, refineLandmarks: false, flipHorizontal: true };

// fit transform every frame so keypoints map exactly
// to the video pixels we're drawing on screen.
let videoScale = 1;
let videoOffsetX = 0;
let videoOffsetY = 0;

function preload() {
  faceMesh = ml5.faceMesh(options);
}

function setup() {
  createCanvas(windowWidth, windowHeight);
  textFont('monospace');
  textSize(14);
  pixelDensity(1);

  video = createCapture(VIDEO, { flipped: true });
  video.size(640, 480);
  video.hide();

  faceMesh.detectStart(video, gotFaces);
  state = 'detecting face';
}

function draw() {
  background(20, 100);

  if (video && video.width > 0) {
    videoScale = max(width / video.width, height / video.height);
    const drawW = video.width * videoScale;
    const drawH = video.height * videoScale;
    videoOffsetX = (width - drawW) / 2;
    videoOffsetY = (height - drawH) / 2;

    //image(video, videoOffsetX, videoOffsetY, drawW, drawH);

    for (let i = 0; i < faces.length; i++) {
      const face = faces[i];
      for (let j = 0; j < face.keypoints.length; j++) {
        const kp = face.keypoints[j];
        const x = videoOffsetX + kp.x * videoScale;
        const y = videoOffsetY + kp.y * videoScale;
```

```
        fill(map(j, 0,face.keypoints.length, 0, 255 ), 255, map(j,
0,face.keypoints.length, 255,0 ));
        noStroke();
        circle(x, y, 10 * sin(frameCount* 0.1 + j)%10);
      }
    }
  }

  noStroke();
  fill(255);
  text(state, 10, height - 10);
}

function gotFaces(results) {
  faces = results;
}
```

handPose image brush

```
let handPose;
let hands = [];

let leftHand = {
  thumb: { x: 0, y: 0 },
  index: { x: 0, y: 0 },
  midpoint: { x: 0, y: 0 },
  isPinching: false,
  distance: 0,
  trail: [],
  detected: false
};

let rightHand = {
  thumb: { x: 0, y: 0 },
  index: { x: 0, y: 0 },
  midpoint: { x: 0, y: 0 },
  isPinching: false,
  distance: 0,
  trail: [],
  detected: false
};

let pinchThreshold = 70;
let trailLength = 10;
```

```
// Initialize hand tracking
function preload() {
  handPose = ml5.handPose({ flipped: true });
}

// Start hand detection with video
function startHandDetection(videoElement) {
  handPose.detectStart(videoElement, gotHands);
}

// Callback for hand detection results
function gotHands(results) {
  hands = results;
}

// Update hand tracking data
function updateHandTracking() {
  // Reset detection
  leftHand.detected = false;
  rightHand.detected = false;

  for (let i = 0; i < hands.length; i++) {
    let hand = hands[i];

    // Get thumb tip and index finger tip
    // HandPose keypoints: 4 = thumb tip, 8 = index finger tip
    let thumbTip = hand.keypoints[4];
    let indexTip = hand.keypoints[8];

    if (!thumbTip || !indexTip) continue;

    // Determine if this is left or right hand
    let handObj;
    if (hand.handedness == "Left") {
      handObj = leftHand;
    } else {
      handObj = rightHand;
    }

    handObj.detected = true;
    handObj.thumb.x = thumbTip.x;
    handObj.thumb.y = thumbTip.y;
    handObj.index.x = indexTip.x;
    handObj.index.y = indexTip.y;

    // Calculate distance and midpoint
```

```
    handObj.distance = dist(handObj.thumb.x, handObj.thumb.y,
handObj.index.x, handObj.index.y);
    handObj.midpoint.x = (handObj.thumb.x + handObj.index.x) / 2;
    handObj.midpoint.y = (handObj.thumb.y + handObj.index.y) / 2;

    // Check if pinching
    handObj.isPinching = handObj.distance < pinchThreshold;

    if (handObj.isPinching) {
      handObj.trail.push({ x: handObj.midpoint.x, y: handObj.midpoint.y });
      if (handObj.trail.length > trailLength) {
        //handObj.trail.shift();
      }
    } else {
      handObj.trail = [];
    }
  }

  // Clear trail if hand not detected
  if (!leftHand.detected) {
    leftHand.isPinching = false;
    leftHand.trail = [];
  }
  if (!rightHand.detected) {
    rightHand.isPinching = false;
    rightHand.trail = [];
  }
}

// Draw all hand keypoints
function drawHandKeypoints() {
  for (let i = 0; i < hands.length; i++) {
    let hand = hands[i];

    for (let j = 0; j < hand.keypoints.length; j++) {
      let kp = hand.keypoints[j];
      if (kp) {
        fill(255, 0, 0, 150);
        noStroke();
        circle(kp.x, kp.y, 8);
      }
    }
  }
}
```

```
function drawHand(hand) {
  push();

  // Draw line between thumb and index
  if (hand.isPinching) {
    stroke(0, 255, 0);
    strokeWeight(4);
  } else {
    stroke(255, 100, 100);
    strokeWeight(2);
  }
  line(hand.thumb.x, hand.thumb.y, hand.index.x, hand.index.y);

  // Draw thumb tip
  fill(255, 200, 0);
  noStroke();
  circle(hand.thumb.x, hand.thumb.y, 15);

  // Draw index finger tip
  fill(0, 200, 255);
  circle(hand.index.x, hand.index.y, 15);

  // Draw midpoint with texture/eraser preview if pinching
  if (hand.isPinching) {
    if (eraseMode) {
      // Show eraser preview
      fill(255, 50, 50, 150);
      stroke(255, 0, 0);
      strokeWeight(2);
      circle(hand.midpoint.x, hand.midpoint.y, brushSize);
      noStroke();
    } else if (selectedTextureIndex >= 0 && textures[selectedTextureIndex])
{
      // Show texture preview
      imageMode(CENTER);
      tint(255, 200);
      image(textures[selectedTextureIndex], hand.midpoint.x,
hand.midpoint.y,
        brushSize, brushSize);
    }
  } else {
    fill(255, 100, 255, 100);
    circle(hand.midpoint.x, hand.midpoint.y, 20);
  }

  // Draw distance text
```

```
  fill(255);
  textSize(12);
  textAlign(CENTER);
  text(int(hand.distance), hand.midpoint.x, hand.midpoint.y - 30);

  pop();
}

// Get number of detected hands
function getHandsCount() {
  return hands.length;
}
```

handPose 3D cube

```
let handPose;
let hands = [];

let leftHand = {
  thumb: { x: 0, y: 0, z: 0 },
  index: { x: 0, y: 0, z: 0 },
  midpoint: { x: 0, y: 0, z: 0 },
  isPinching: false,
  distance: 0,
  detected: false,
};

let rightHand = {
  thumb: { x: 0, y: 0, z: 0 },
  index: { x: 0, y: 0, z: 0 },
  midpoint: { x: 0, y: 0, z: 0 },
  isPinching: false,
  distance: 0,
  detected: false,
};

let pinchThreshold = 50;

// Initialize hand tracking
function preload() {
  handPose = ml5.handPose({ flipped: true });
}

// Start hand detection with video
```

```
function startHandDetection(videoElement) {
  handPose.detectStart(videoElement, gotHands);
}

// Callback for hand detection results
function gotHands(results) {
  hands = results;
}

// Update hand tracking data
function updateHandTracking() {

  leftHand.detected = false;
  rightHand.detected = false;


  for (let i = 0; i < hands.length; i++) {
    let hand = hands[i];

    // Get thumb tip and index finger tip
    // HandPose keypoints: 4 = thumb tip, 8 = index finger tip
    let thumbTip = hand.keypoints[4];
    let indexTip = hand.keypoints[8];

    if (!thumbTip || !indexTip) continue;

    // Determine if this is left or right hand
    let handObj;
    if (hand.handedness == "Left") {
      handObj = leftHand;
    } else {
      handObj = rightHand;
    }

    handObj.detected = true;
    handObj.thumb.x = thumbTip.x;
    handObj.thumb.y = thumbTip.y;
    handObj.index.x = indexTip.x;
    handObj.index.y = indexTip.y;

    // Calculate distance and midpoint
    handObj.distance = dist(handObj.thumb.x, handObj.thumb.y,
handObj.index.x, handObj.index.y);
    handObj.midpoint.x = (handObj.thumb.x + handObj.index.x) / 2;
    handObj.midpoint.y = (handObj.thumb.y + handObj.index.y) / 2;
```

```
    // Check if pinching
    handObj.isPinching = handObj.distance < pinchThreshold;

  }

}
// Draw all hand keypoints
function drawHandKeypoints() {
  for (let i = 0; i < hands.length; i++) {
    let hand = hands[i];

    for (let j = 0; j < hand.keypoints.length; j++) {
      let kp = hand.keypoints[j];
      if (kp) {
        fill(255, 255, 255, 150);
        noStroke();
        circle(kp.x, kp.y, 8);
      }
    }
  }
}

function drawHand(hand) {
  stroke(255, 255, 255);
  strokeWeight(2);
  line(hand.thumb.x, hand.thumb.y, hand.index.x, hand.index.y);

  // Draw thumb tip
  fill(255, 100, 0);
  circle(hand.thumb.x, hand.thumb.y, 15);

  // Draw index finger tip
  fill(0, 200, 255);
  circle(hand.index.x, hand.index.y, 15);

  // Draw midpoint between thumb and index finger tip
  fill(100, 255, 0);
  circle(hand.midpoint.x, hand.midpoint.y, 20);


}

function getHandsCount() {
  return hands.length;
}
```